MINISTRY OF EDUCATION, SINGAPORE
in collaboration with
CAMBRIDGE ASSESSMENT INTERNATIONAL EDUCATION
General Certificate of Education Advanced Level
Higher 2

COMPUTING

9569/02

Paper 2 (Lab-based)

October/November 2025

3 hours

Additional Materials: Electronic version of `comment.txt` data file
Electronic version of `post.txt` data file
Electronic version of `records.txt` data file
Electronic version of `rooms.txt` data file
Electronic version of `user.txt` data file
Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [ ] at the end of each question or part question.
The total number of marks for this paper is 100.

This document consists of **11** printed pages, **1** blank page and **1** Insert.

Singapore Examinations and Assessment Board

Cambridge Assessment International Education

Instruction to candidates:

Your program code and output for each of Task 1 to 4 should be saved in a single `.ipynb` file. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

Task 1

Name your Jupyter Notebook as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

A denary number can be converted to binary using the division by 2 method. The denary number is divided by 2 repeatedly until it becomes 0. After each division, the remainder of 1 or 0 is stored. These remainders are written in reverse order to create the binary number. For example, converting the denary number 28 to binary:

28 / 2 = 14 remainder 0
14 / 2 = 7 remainder 0
7 / 2 = 3 remainder 1
3 / 2 = 1 remainder 1
1 / 2 = 0 remainder 1
28 denary is 11100 in binary.

A denary number can be converted to binary using the division by 2 method and a stack.

The stack can store any number of integer data items using a 1-dimensional list. The top of stack pointer stores the index of the next space to store data in the stack.

The data is **not** deleted when it is popped from the stack; the top of stack pointer is changed to point to the next space to store data.

The program must only use local variables; any data required by a function needs to be passed as parameters and returned (where appropriate).

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program code
```

```
Output:
```

Task 1.1

The function `push()` takes the stack, the top of stack pointer and the data item to push onto the stack as parameters. The function stores the data item in the stack and updates the top of stack pointer. The function returns the top of stack pointer. If the stack is full, the length of the list should be increased.

Write program code for the function `push()` [4]

Task 1.2

The function `pop()` takes the stack and the top of stack pointer as parameters. The data item at the top of the stack is returned along with the updated top of stack pointer.

The data is **not** deleted when it is popped. The top of stack pointer is changed to point to the new top of stack; this allows data to be overwritten when new data is pushed onto the stack.

If the stack is empty, the function returns `False`

Write program code for the function `pop()` [4]

Task 1.3

The function `main()` declares a 1-dimensional list to represent the stack and initialises the top of stack pointer to 0.

The function takes a positive integer as input from the user or as a parameter.

The function only outputs to the screen and does **not** return any data.

If the number input is 0, the program outputs the binary value 0.

If the number input is greater than 0, the input is repeatedly divided by 2 until it reaches 0; each remainder is pushed onto the stack using the function `push()` you created in Task 1.1. Each digit is then popped from the stack to create the binary number using the function `pop()` you created in Task 1.2. The binary number is then output on one line.

Write program code for `main()` and call this function. [10]

Test your program with the following inputs:

Test 1: 0

Test 2: 22

Test 3: 46967

Show the output for each test. [2]

Save your Jupyter Notebook for Task 1.

Task 2

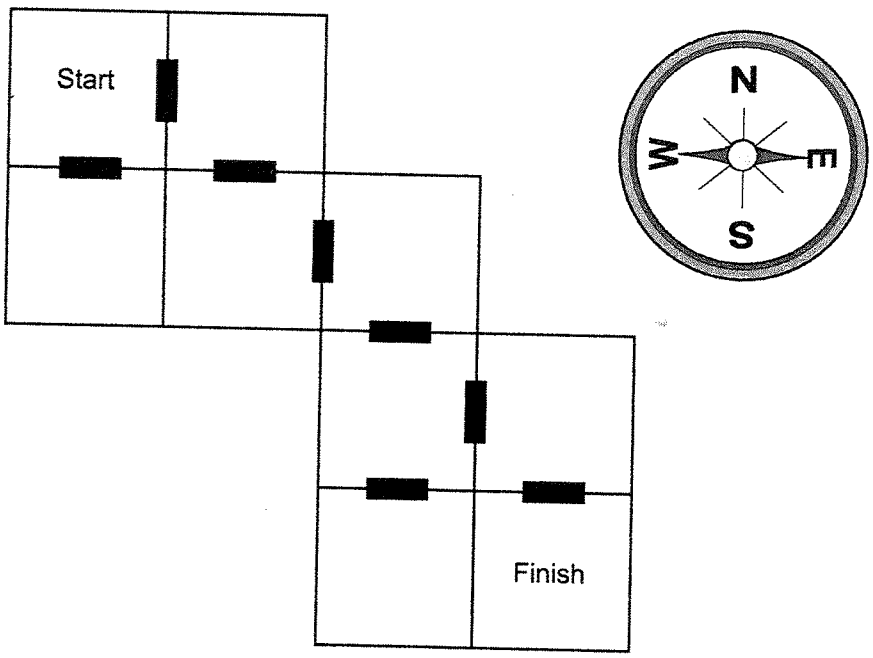
Name your Jupyter Notebook as:

`TASK2_<your name>_<centre number>_<index number>.ipynb`

A prototype for a computer game needs to be created using Object-Oriented Programming.

The game has a player who has to move through a series of rooms. Each room has at least one door to one other room. In all rooms apart from the Start room, a user has to answer a question correctly before he can move to the room he has chosen.

An example layout for a series of rooms is shown. This layout is an example and does **not** match the layout for the game you will be programming.



Each room has four positions where a door can be situated: North, South, East and West. Not all rooms have a door in each position. In the example diagram, each room is represented by a square and each door is represented by a solid rectangle between the two rooms. For example, the Start room only has doors to the East and the South.

Each room is given a number; the Start room is room number 0.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code

        Output:
```

Task 2.1

The class `Room` contains four attributes:

- `north_exit` – an integer for the room number that the North door leads to, or –1 if there is no door.
- `east_exit` – an integer for the room number that the East door leads to, or –1 if there is no door.
- `south_exit` – an integer for the room number that the South door leads to, or –1 if there is no door.
- `west_exit` – an integer for the room number that the West door leads to, or –1 if there is no door.

The class `Room` has the following methods:

- A constructor that takes four integers for `north_exit`, `east_exit`, `south_exit` and `west_exit` as parameters. Each parameter is assigned to the appropriate attribute.
- `get_exit()` that takes a character as a parameter (`'N'`, `'S'`, `'E'`, or `'W'`) to represent the door selected and returns the attribute value for that door.

Write program code to declare the class `Room` and its methods. [5]

Three classes inherit from `Room`: `Start`, `Finish` and `Normal`

`Start` is the first room where the game begins and also contains:

- the attribute `start_text` that is output at the beginning of the game. This value is initialised in the constructor from a parameter
- the method `get_message()` that returns the string in `start_text` and a statement for each direction where there is a door from that room, for example 'There is a door to the North.' The method also returns the string `"Start"` (for example as part of a tuple) to indicate that this is the Start room.

Write program code to declare the class `Start` and its methods. [5]

`Finish` is the room where the game ends and also contains:

- the attribute `finish_text` that is output at the end of the game. This value is initialised in the constructor from a parameter
- the method `get_message()` that returns `finish_text` and the string `"Finish"` (for example as part of a tuple) to indicate that the game is finished.

Write program code to declare the class `Finish` and its methods. [2]

`Normal` is a room that is **not** the Start room and is **not** the Finish room. The class `Normal` also contains:

- the attributes `question` and `answer`. The value of each is initialised in the constructor from its parameter
- the method `get_question()` that returns the data in `question`
- the method `check_answer()` that compares its parameter (the user's answer) to `answer` and returns `True` if it matches and `False` if it does **not** match
- the method `get_message()` that returns a string describing all of the doors from that room, for example 'There is a room to the North. There is a room to the West.' The method also returns the string `"Normal"` (for example as part of a tuple) to indicate that this is a normal room.

Write program code to declare the class `Normal` and its methods. [4]

Task 2.2

The text file `rooms.txt` stores data about the rooms in the game. Each row of data is a separate room.

The values in each row are in the order:

- room type: Start, Finish or Normal
- the room number that can be accessed to the North of the room, or –1 if there is no door
- the room number that can be accessed to the East of the room, or –1 if there is no door
- the room number that can be accessed to the South of the room, or –1 if there is no door
- the room number that can be accessed to the West of the room, or –1 if there is no door
- the start message if the room is of type `Start`
  or the finish message if the room is of type `Finish`
  or the question if the room is of type `Normal`
- the answer to the question if the room is of type `Normal`

The rooms are read in the order they appear in the file and stored in a 1-dimensional list. For example: the room in the first row of the file is stored in index 0; the room in the second row of the file is stored in index 1.

Write program code for a function to:

- read the data from the text file
- create an object of the appropriate class for each room
- store the rooms in a 1-dimensional list
- return the 1-dimensional list.

[7]

Task 2.3

When the game is played, the message in the Start room is output which includes the position of the door(s).

The user inputs the door to go through until he provides a valid door. The message for the new room is output which includes the position of the door(s) in that room. When the user selects a door, the question for the room he is in is output. The user needs to enter his answer until he provides the correct answer.

The process repeats until the user reaches the Finish room. When the user reaches the Finish room, the message for the room is output and the game ends.

Write program code for a function to allow a user to play the game and call this function. [13]

Test your program with the following inputs:

input 1: N

input 2: E

input 3: 00

input 4: FF

input 5: N

input 6: 0

input 7: N

input 8: 170

input 9: E

input 10: 0

input 11: N

input 12: 2

input 13: E

input 14: 4

[2]

Save your Jupyter Notebook for Task 2.

[Turn over

Task 3

Name your Jupyter Notebook as:

`TASK3_<your name>_<centre number>_<index number>.ipynb`

A social media website allows users to create accounts. Users can create posts; these are messages that are displayed to other users. Users can comment on posts.

A database stores data about users, the posts and comments on each post from the social media website.

Three tables in the database are shown:

`User (Username, Blocked)`

`Post (PostID, PostText, DateTimePosted, Username)`

`Comment (CommentID, PostID, Username, TheText, DateTimeCommented)`

A primary key is indicated by underlining one or more attributes. Foreign keys are indicated by using a dashed underline.

The database is being created and tested using sample data which is given in the following text files: `user.txt`, `post.txt` and `comment.txt`

The field `Blocked` in the table `User` stores `1` if the user is currently blocked and stores `0` if the user is **not** blocked.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 3.1
        Program code

        Output:
```

Task 3.1

Write a Python program that uses SQL to create a database with the **three** tables: `User`, `Post` and `Comment`

Define the primary and foreign keys for each table.

[6]

Task 3.2

Write a Python program that uses SQL to insert the data from each of the **three** text files into the appropriate fields in each database table. [4]

Run your program. [1]

Task 3.3

Write a Python function that:

- uses SQL to search for all comments by a user who is blocked
- outputs each returned username and comment on a new line.

[5]

Run your program.

[1]

Save your Jupyter Notebook for Task 3.

Task 4

Name your Jupyter Notebook as:

`TASK4_<your name>_<centre number>_<index number>.ipynb`

A computer program prototype is being developed to store records in a hash table.

Each record has a key field (positive integer) and a string data item.

The hash table is set up as a list, initialised with capacity for 100 records.

The hashing algorithm is: `hash(k) = k mod s`

`k` represents the key field and `s` represents the maximum size of the hash table.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]:  #Task 4.1
         Program code

         Output:
```

Task 4.1

Write program code to initialise the global list `hash_table` with 100 empty records. [1]

Task 4.2

Write program code for the function `calculate_hash()` to take a key field as a parameter and return the calculated hash value using the given hashing algorithm. [2]

Task 4.3

The hash table uses linear probing when there is a collision. Linear probing is when the next index is checked to identify whether the index is free. This repeats until either a free index is found, or all indices have been checked and the list is full.

Write program code for the function `insert()` to:

- take a key field and string data item as parameters
- calculate the index using the `calculate_hash()` function
- store the key field and string data item in the index if available, otherwise use linear probing to find the next free index
- return `True` if the record was successfully stored or `False` if the hash table is full.

[6]

Task 4.4

The text file `records.txt` stores 70 key fields and the associated string data item. For this prototype, the string data item is a single word. The key field and string data item are separated by a space.

Write program code for the function `read_data()` to:

- read each key field and string data item from the text file
- call the function `insert()` with each key field and string data item
- output an appropriate message if any record cannot be stored in the hash table.

[4]

Call the function `read_data()` and then output the contents of the hash table. [1]

Task 4.5

Write program code for the function `find_record()` to:

- take a key field as a parameter
- use its hash value to return the string data item for that record or return `False` if the record is **not** stored in the hash table.

[3]

Task 4.6

Write program code to call the function `read_data()` again and call the function `find_record()` with the key field `475`
If the record is found, output the string data item in an appropriate message. If the record is **not** found, output `"Record not found"`
[1]

Test your program.
[1]

Save your Jupyter Notebook for Task 4.

BLANK PAGE